

Programação Orientada a Objetos em Java

Prof. Dani Juliano Czelusniak

Atualizado em 2026

1 Análise Teórica: O Paradigma Orientado a Objetos

A transição do pensamento estruturado para a Programação Orientada a Objetos (POO) exige uma mudança na forma como modelamos problemas. Na POO, o sistema é concebido como um ecossistema de **objetos** que interagem. O código de uma aplicação orientada a objetos consiste primariamente em um conjunto de **classes**, enquanto a execução dessa aplicação é composta por um conjunto de objetos se comunicando.

1.1 Classes e Objetos

Uma **classe** é um gabarito, um molde que define a estrutura (estado) e o comportamento (operações) de um tipo específico de entidade no sistema. Um **objeto** é uma instância concreta gerada a partir dessa classe, alocada na memória do computador.

Uma classe é composta por dois pilares fundamentais:

- **Atributos (Estado):** São as variáveis definidas no escopo da classe. Definem a estrutura dos objetos. Cada objeto instanciado possui os seus próprios valores para esses atributos.
- **Métodos (Comportamento):** São as funções aplicáveis a todo objeto da classe. Determinam como o objeto pode ser manipulado ou como seu estado pode ser alterado ou consultado. Um atributo pode ser acessado por todos os métodos internos da classe.

Para ilustrar, considere um sistema de mobilidade (como um aplicativo de transporte) conforme figura abaixo. Um aplicativo de mobilidade que disponibiliza um conjunto de carros para fazer o deslocamento de pessoas. O aplicativo mantém os seguintes dados sobre cada carro:

- identificação única: um número inteiro
- posição no mapa: coordenadas (x, y)
- velocidade: em Km/h
- direção de movimento: Norte, Sul, Leste ou Oeste

As operações aplicáveis a um carro são:

- atualizar as coordenadas

- atualizar a velocidade
- atualizar a direção do movimento (Norte, Sul, Leste, Oeste)
- estimar o tempo para chegar em certo ponto do mapa

A Figura 1 ilustra visualmente a representação dos objetos carros dentro do sistema de mobilidade urbana, evidenciando como múltiplas instâncias coexistem no mapa de forma independente, cada qual mantendo seus próprios dados dinâmicos de identificação, posição cartesiana, velocidade e vetor de deslocamento.

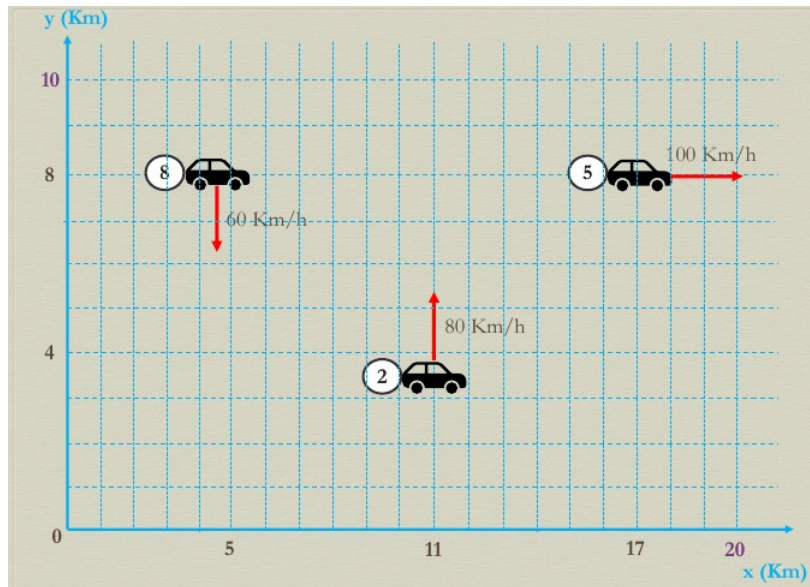


Figura 1: Objetos Carros.

Podemos abstrair a classe **Carro** com os atributos: **numero** de identificação, **coordenada_x**, **coordenada_y**, **velocidade** e **direcao**. Seus métodos incluiriam **atualizar_coordenadas()**, **atualizar_velocidade()** e **estimar_tempo()**. Durante a simulação, múltiplos objetos do tipo **Carro** coexistem, cada um em sua própria posição e velocidade.

A Figura 2 apresenta o diagrama estrutural reduzido da classe **Carro**, detalhando o molde arquitetural utilizado para gerar as instâncias individuais e sintetizando visualmente o mapeamento entre os atributos essenciais e as operações de manipulação de estado correspondentes.

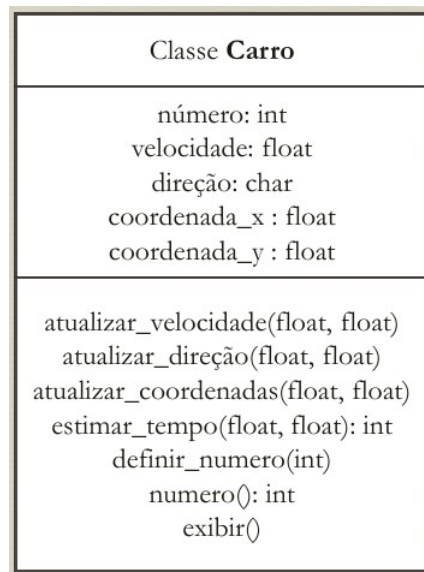


Figura 2: Diagrama da classe Carro com tamanho reduzido.

1.2 Encapsulamento e Qualificadores de Visibilidade

Um dos princípios fundamentais da POO é o **Encapsulamento**. Ele dita que uma classe deve ocultar seus detalhes de implementação (seus dados sensíveis) e expor apenas uma interface pública segura para o mundo externo. Isso previne corrupção de estado e alto acoplamento.

Para aplicar o encapsulamento, o Java utiliza qualificadores de proteção e visibilidade:

- **public**: Permite acesso irrestrito a partir de qualquer classe do projeto.
- **private**: O acesso é estritamente restrito aos métodos da própria classe.
- **protected**: Permite acesso dentro do mesmo pacote e em classes herdeiras (subclasses).
- **package** (padrão): Acesso restrito a classes que pertencem ao mesmo pacote (ausência de modificador).

Geralmente, a regra de ouro do encapsulamento em Java é: **atributos devem ser declarados como private** e o acesso a eles, quando necessário, deve ser feito através de **métodos public** (como *getters*, *setters* ou métodos de negócio específicos).

A Figura 3 demonstra a conceituação da interface pública de uma classe, destacando quais métodos e pontos de acesso são expostos de forma controlada para o restante do sistema, resguardando os dados internos sob os princípios do encapsulamento.

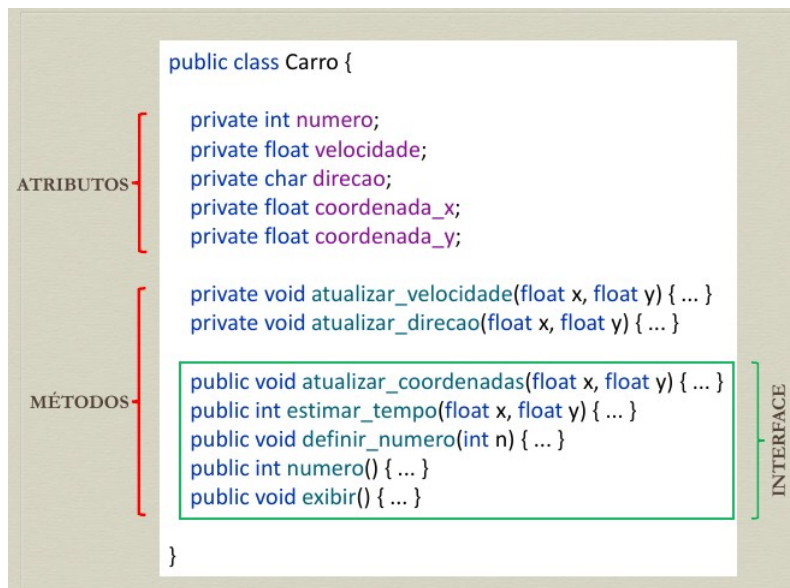


Figura 3: Exemplificação da Interface da Classe.

O conjunto de métodos de uma classe especificados como acessíveis por outras classes constitui a **interface** dessa classe.

A Figura 4 exibe a implementação detalhada do método responsável por atualizar a velocidade do carro, ilustrando como a lógica interna processa os parâmetros recebidos para modificar de maneira segura o estado dinâmico da instância.



Figura 4: Método que Atualiza a Velocidade do Carro.

A Figura 5 ilustra a estrutura de código do método encarregado de atualizar a direção do movimento do carro, detalhando a manipulação da orientação geográfica do veículo (Norte, Sul, Leste ou Oeste) dentro do escopo do objeto.

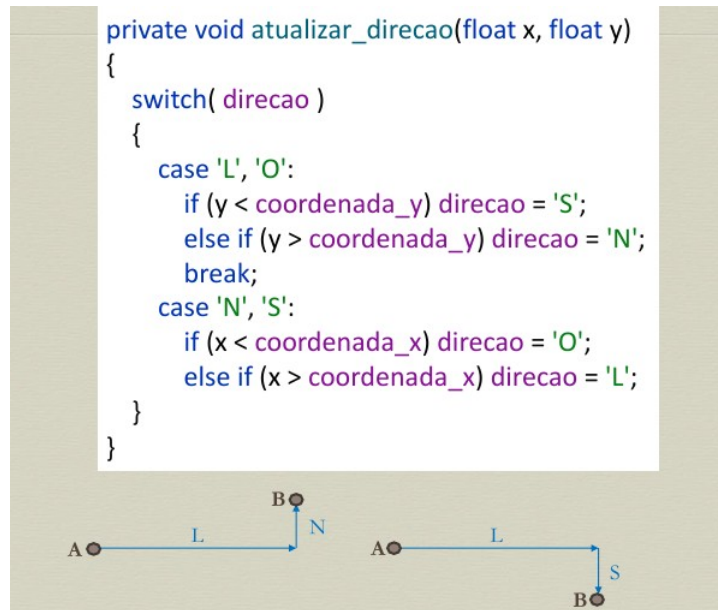


Figura 5: Método que Atualiza a Direção do Carro.

A Figura 6 apresenta o procedimento computacional utilizado para atualizar as coordenadas cartesianas (x, y) do carro no mapa, refletindo o deslocamento espacial contínuo da instância simulada no sistema.

```
public void atualizar_coordenadas(float x, float y)
{
    atualizar_velocidade(x, y);
    atualizar_direcao(x, y);
    coordenada_x = x;
    coordenada_y = y;
}
```

Figura 6: Método que Atualiza as Coordenadas (x,y) do Carro.

A Figura 7 detalha o método de negócio voltado para estimar o tempo de chegada do carro a um determinado ponto do mapa, evidenciando como o objeto processa seus dados atuais de posição e velocidade para retornar uma informação útil ao usuário.

```

public int estimar_tempo(float x, float y)
{
    int tempo_estimado = 0; // em segundos
    float delta_x = Math.abs(x - coordenada_x);
    float delta_y = Math.abs(y - coordenada_y);
    float distancia_a_percorrer = delta_x + delta_y;
    tempo_estimado = (int) (distancia_a_percorrer / velocidade * 3600);
    return tempo_estimado;
}

```

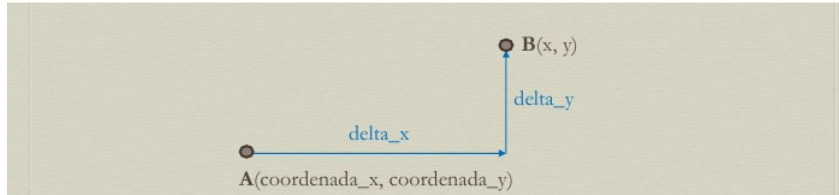


Figura 7: Método que Atualiza a Estimativa de Tempo do Carro.

A Figura 8 exibe o método destinado a definir ou atribuir o número de identificação única do carro, assegurando que cada objeto instanciado possua uma chave de diferenciação consistente logo em sua inicialização ou parametrização.

```

public void definir_numero(int n)
{
    numero = n;
}

public int numero()
{
    return numero;
}

```

Figura 8: Método que Define o Número do Carro.

A Figura 9 demonstra o método utilitário voltado para a exibição textual do estado atual do carro, compilando as informações dos atributos privados e formatando-os para visualização direta no console ou interface de teste.

```

public void exibir()
{
    System.out.println("Carro " + numero + ":");

    System.out.println(String.format("Coordenadas atuais: (%.2f; %.2f)",
        coordenada_x, coordenada_y));

    System.out.println(String.format("Velocidade: %.2f Km/h",
        velocidade));

    System.out.println("Direção: " + direcao);

    System.out.println("-----");
}

```

Figura 9: Método que Exibe o Carro.

```

1 class ContaBancaria {
2     // Atributo protegido e oculto
3     private double saldo;
4
5     // Construtor
6     public ContaBancaria(double saldoInicial) {
7         if(saldoInicial < 0) {
8             throw new IllegalArgumentException("Saldo inicial
9         invalido!");
10        }
11        this.saldo = saldoInicial;
12    }
13
14    // Metodo de negocio expondo alteracao segura de estado
15    public void depositar(double valor) {
16        if(valor <= 0) {
17            throw new IllegalArgumentException("Valor de deposito
18        invalido!");
19        }
20        this.saldo += valor;
21    }
22
23    // Metodo de consulta (Getter)
24    public double getSaldo() {
25        return this.saldo;
26    }
27 }
28
29 public class TesteConta {
30     public static void main(String[] args) {
31         try {
32             ContaBancaria cb = new ContaBancaria(100.0);
33             System.out.println("Saldo inicial: R$ " + cb.getSaldo());
34         }
35     }
36 }

```

```

33         cb.depositar(50.0);
34         System.out.println("Saldo apos deposito: R$ " + cb.
getSaldo());
35
36         // Forcando erro com deposito negativo para acionar
validacao
37         System.out.println("Tentando depositar valor negativo..."
);
38         cb.depositar(-20.0);
39     } catch (IllegalArgumentException e) {
40         System.out.println("Erro na operacao: " + e.getMessage())
;
41     }
42 }
43 }

```

Listing 1: Exemplo de Encapsulamento com Validação e Teste prático

2 O Método Construtor e a Referência `this`

2.1 A Referência `this`

A palavra-chave `this` em Java é uma referência interna obrigatória para acessar o próprio objeto alvo da execução. O uso mais clássico do `this` é a **resolução de ambiguidade** entre os atributos da classe e os parâmetros de um método. Quando o parâmetro de um método (ex: `int idade`) tem exatamente o mesmo identificador do atributo da classe (`private int idade`), a linguagem prioriza a variável mais local (o parâmetro). Para forçar o compilador a entender que queremos gravar a informação no atributo do objeto, utilizamos `this.idade = idade;`.

2.2 Métodos Construtores

A criação de um objeto (*instanciação*) ocorre por meio da palavra reservada `new`. Internamente, o `new` executa o **método construtor** da classe. As características de um construtor são:

1. Possui nome **idêntico** ao nome da classe.
2. Não possui tipo de retorno na sua assinatura (nem mesmo `void`).
3. Geralmente, possui visibilidade `public`.

Se o programador não declarar um construtor explicitamente, a classe assume o chamado **Construtor Default** (sem parâmetros), inicializando atributos numéricos com 0 e referências com `null`. No entanto, se um construtor parametrizado for criado (ex: `Imovel(int codigo, double area)`), o construtor default deixa de existir automaticamente, obrigando quem for instanciar a classe a fornecer os dados requeridos, garantindo que o objeto seja criado em um estado válido e consistente num único passo.

2.3 Sobrecarga de Construtores

Uma classe pode declarar múltiplos construtores, permitindo flexibilidade na instanciação. Isso se chama *sobrecarga* (*overload*). A única exigência do compilador é que cada construtor possua uma **assinatura diferente** (uma quantidade ou tipos de parâmetros distintos).

```
1 class Pessoa {
2     private String nome;
3     private int idade;
4
5     // Construtor Completo
6     public Pessoa(String nome, int idade) {
7         this.nome = nome;
8         this.idade = idade;
9     }
10
11    // Construtor Parcial (assume idade desconhecida como -1)
12    public Pessoa(String nome) {
13        this.nome = nome;
14        this.idade = -1;
15    }
16
17    // Construtor Default explicitamente recriado
18    public Pessoa() {
19        this.nome = "Desconhecido";
20        this.idade = -1;
21    }
22
23    public void exibirInfo() {
24        System.out.println("Nome: " + this.nome + " | Idade: " + this
25        .idade);
26    }
27 }
28
29 public class TestePessoa {
30     public static void main(String[] args) {
31         // Criando os objetos invocando diferentes construtores
32         Pessoa p1 = new Pessoa("Ana", 28);
33         Pessoa p2 = new Pessoa("Beto");
34         Pessoa p3 = new Pessoa();
35
36         System.out.println("Estado dos objetos criados:");
37         p1.exibirInfo();
38         p2.exibirInfo();
39         p3.exibirInfo();
40     }
41 }
```

Listing 2: Sobrecarga de Construtores com Classe de Execução

3 Composição: Modelando Objetos Complexos

Um dos recursos mais poderosos da Orientação a Objetos é a capacidade de um objeto conter outros objetos como atributos, técnica conhecida como **Composição**.

No desenvolvimento geométrico, por exemplo, a representação de retas e polígonos no plano cartesiano demonstra perfeitamente a composição. Um `Segmento_de_Reta` é, matematicamente, definido pela ligação entre dois pontos. Logo, a classe `Segmento_de_Reta` deve possuir dois atributos do tipo `Ponto` (`p1` e `p2`). Por conseguinte, um `Triangulo` seria composto por três `Pontos` e delimitado graficamente pela construção de três segmentos iterados em sua renderização.

```
1 // Objeto base
2 class Ponto {
3     private double x, y;
4     public Ponto(double x, double y) { this.x = x; this.y = y; }
5     public String texto() { return "(" + x + ", " + y + ")"; }
6 }
7
8 // Objeto complexo composto por Pontos
9 class Segmento_de_Reta {
10     private Ponto p1, p2; // Composicao
11
12     // Qualquer metodo pode receber outros objetos como parametro
13     public Segmento_de_Reta(Ponto p1, Ponto p2) {
14         this.p1 = p1;
15         this.p2 = p2;
16     }
17
18     public void desenhe() {
19         System.out.println("Reta entre " + p1.texto() + " e " + p2.
20         texto());
21     }
22 }
23
24 public class TesteGeometria {
25     public static void main(String[] args) {
26         // Instanciando os objetos base (Pontos)
27         Ponto pA = new Ponto(2.5, 3.0);
28         Ponto pB = new Ponto(7.0, 8.5);
29
30         // Passando a referencia dos pontos para compor a reta
31         Segmento_de_Reta reta = new Segmento_de_Reta(pA, pB);
32
33         // Desenhando
34         reta.desenhe();
35     }
36 }
```

Listing 3: Composição de Elementos Geométricos e Simulação

3.1 Geradores de Objetos (Factories)

Em arquiteturas de software sofisticadas, a criação de múltiplos objetos complexos (como Triângulos e Quadriláteros) frequentemente não é feita diretamente no método principal. Utiliza-se a abordagem de criar um intermediário. Uma classe `Gerador_de_Poligonos`, por exemplo, encapsula as regras de instanciação. Isso não só deixa o código cliente (projetista) mais limpo, como abre margem para validações centralizadas, auditoria (contagem de formas geradas) e segurança na alocação da memória.

4 Interação e Mensageria entre Objetos

Um software orientado a objetos **não é** uma execução isolada de procedimentos globais. Ao contrário, o sistema ganha vida através da **interação (mensageria)** entre seus objetos instanciados. A interação ocorre estritamente quando um objeto A invoca os métodos (preferencialmente os públicos) do objeto B.

Para mapear e arquitetar essas interações antes da codificação, o mercado utiliza a **UML** (*Unified Modeling Language*). O *Diagrama de Classes* descreve o projeto estrutural (atributos, métodos e multiplicidade), enquanto o *Diagrama de Sequência* detalha o eixo temporal de comunicação.

Como exemplo, na arquitetura do clássico Jogo da Velha:

- `JogoDaVelha` (main): Inicializa o contexto, cria o Tabuleiro e os Jogadores.
- `Partida`: Mantém o laço principal (`executar()`) de turnos.
- `Jogador`: Aciona o método de `jogar()`, que enviará uma mensagem de `marcar(x, y)` para o Tabuleiro.
- `Tabuleiro`: Valida a consistência das coordenadas e aciona internamente o método de `verificar_jogada()` para determinar estado de vitória.

5 Estudos de Caso e Exercícios Resolvidos

Para solidificar a junção do uso de composição, construtores robustos e tratamento de exceções, observe o desenvolvimento dos cenários práticos a seguir.

5.1 Caso 1: Sistema de Vendas (Composição Simples)

Problema: Implemente um sistema composto por `Produto` e `ItemPedido`. O item de pedido deve calcular seu subtotal invocando informações do produto. Utilize validação para barrar quantidades negativas. Crie a classe de teste principal para rodar a simulação e instanciar os objetos.

```
1 class Produto {
2     private int codigo;
3     private double preco;
4
5     public Produto(int codigo, double preco) {
6         this.codigo = codigo;
7         this.preco = preco;
```

```

8      }
9      public double getPreco() { return this.preco; }
10 }
11
12 class ItemPedido {
13     private Produto produto; // Composicao
14     private int quantidade;
15
16     public ItemPedido(Produto produto, int quantidade) throws
Exception {
17         if (quantidade <= 0) {
18             throw new Exception("Quantidade deve ser maior que zero!"
19 );
20         }
21         this.produto = produto;
22         this.quantidade = quantidade;
23     }
24
25     public double calcularSubtotal() {
26         return this.quantidade * this.produto.getPreco(); //
Interacao
27     }
28 }
29
30 public class TesteVendas {
31     public static void main(String[] args) {
32         Produto p1 = new Produto(101, 25.50);
33
34         try {
35             // Simulando uma compra valida
36             ItemPedido item = new ItemPedido(p1, 4);
37             System.out.println("Subtotal da compra: R$ " + item.
calcularSubtotal());
38
39             // Forcando o disparo da validacao (erro)
40             ItemPedido itemInvalido = new ItemPedido(p1, -2);
41
42         } catch (Exception e) {
43             System.out.println("Falha na compra: " + e.getMessage());
44         }
45     }

```

Listing 4: Composição em Vendas com Classe de Simulação

5.2 Caso 2: Encapsulamento Avançado em Veículos

Problema: Modele uma Motocicleta para turismo onde o estado do motor (ligado/desligado) e a velocidade precisem ser rigorosamente controlados. O piloto só pode alterar a velocidade se o motor estiver em funcionamento, garantindo a coesão das regras de negócio (encapsulamento total).

```

1 class Motocicleta {
2     private String modelo;
3     private boolean motorLigado;
4     private int velocidadeAtual;
5     private int limiteVelocidade;
6
7     public Motocicleta(String modelo, int limiteVelocidade) {
8         this.modelo = modelo;
9         this.limiteVelocidade = limiteVelocidade;
10        this.motorLigado = false; // Estado inicial consistente
11        this.velocidadeAtual = 0;
12    }
13
14    public void darPartida() {
15        this.motorLigado = true;
16        System.out.println("Motor da " + this.modelo + " ligado.");
17    }
18
19    public void acelerar(int incremento) {
20        if (!this.motorLigado) {
21            System.out.println("Erro: Nao e possivel acelerar com o
22            motor desligado.");
23            return; // Interrompe o metodo para proteger o estado
24        }
25
26        this.velocidadeAtual += incremento;
27
28        if (this.velocidadeAtual > this.limiteVelocidade) {
29            this.velocidadeAtual = this.limiteVelocidade; // Trava de
30            seguranca
31        }
32
33        System.out.println("Acelerando... Velocidade: " + this.
34        velocidadeAtual + " km/h");
35    }
36 }
37
38 public class TesteMotocicleta {
39     public static void main(String[] args) {
40         Motocicleta moto = new Motocicleta("Custom 1200cc", 180);
41
42         moto.acelerar(20); // Falha, motor desligado
43         moto.darPartida(); // Altera o estado
44         moto.acelerar(50); // Sucesso
45     }
46 }

```

Listing 5: Encapsulamento de Estado em Motocicleta

5.3 Caso 3: Interação Complexa no Ambiente Acadêmico

Problema: Simule a interação entre um Professor e um Estudante. O objeto da classe Professor deve realizar uma operação que modifica ativamente o estado do objeto Estudante (atribuir uma nota), demonstrando a mensageria direta entre diferentes entidades do sistema.

```
1 class Estudante {
2     private String nome;
3     private double notaFinal;
4
5     public Estudante(String nome) {
6         this.nome = nome;
7         this.notaFinal = 0.0;
8     }
9
10    // Metodo protegido logicamente: so aceita notas de 0 a 10
11    public void registrarNota(double nota) {
12        if (nota >= 0.0 && nota <= 10.0) {
13            this.notaFinal = nota;
14        } else {
15            System.out.println("Erro: Nota fora do escopo permitido."
16        );
17        }
18    }
19
20    public String getNome() { return this.nome; }
21    public double getNotaFinal() { return this.notaFinal; }
22 }
23
24 class Professor {
25     private String nome;
26
27     public Professor(String nome) {
28         this.nome = nome;
29     }
30
31    // O metodo recebe OUTRO objeto como parametro para interagir com
32    // ele
33    public void avaliarEstudante(Estudante aluno, double nota) {
34        System.out.println("Prof. " + this.nome + " esta avaliando "
35        + aluno.getNome());
36        // Enviando mensagem para o objeto Estudante
37        aluno.registrarNota(nota);
38    }
39 }
40
41 public class TesteAcademico {
42     public static void main(String[] args) {
43         Professor prof = new Professor("Dani Juliano");
44         Estudante est = new Estudante("Carlos Andrade");
```

```
43      // Interacao ocorrendo
44      prof.avaliarEstudante(est, 9.5);
45
46      System.out.println("Nota do " + est.getNome() + ": " + est.
47      getNotaFinal());
48  }
```

Listing 6: Mensageria entre Professor e Estudante